

PROGRAMACIÓN AVANZADA

Clase de utilidad y Enum



Ing. Matias Macrino

CLASE DE UTILIDAD

String

CLASE STRING

► Clase String:

- En Java las cadenas de caracteres no se corresponden con ningún tipo de **dato primitivo**, sino que son objetos pertenecientes a **java.lang.String**.
- Dicha clase proporciona una amplia variedad de métodos que permite realizar las operaciones de manipulación y tratamiento de cadenas de caracteres, habituales en un programa.

CLASE STRING

► Clase String:

- Para crear un objeto **String** podemos seguir el procedimiento general de creación de objetos en java, con el operador “**new**”.
Pero dada la amplia utilización de este objeto dentro de un programa, Java permite crear y asignar un objeto **String** a una variable de la misma forma que se hace con cualquier tipo primitivo.

```
String str = new String("Texto de prueba");  
String str = "Texto de prueba";
```

- Una vez creado el objeto, el mismo puede utilizarse para acceder a los métodos definidos en la clase **String**.

CLASE STRING

- ▶ Clase String:

- ▶ Principales Métodos:

- ▶ Int **length()**. Devuelve el numero de caracteres de la cadena sobre la que se aplica.
- ▶ boolean **equals(Obj String)**. Compara una cadena con otra, haciendo distinción entra mayúsculas y minúsculas.
- ▶ boolean **equalsIgnoreCase(Obj String)**. Igual que **equals**, solo que no hace distinción entre mayúsculas y minúsculas.
- ▶ String **substring(int inicio, int fin)**. Devuelve una porción de la cadena que se aplica.
- ▶ Static **String valueOf(Obj dato_primitivo)**. Método estático que devuelve como cadena el valor pasado como parámetro.
- ▶ String[] **split (String regex)**. Devuele un **array** de **String** resultante de descomponer una cadena de texto en subcadenas.

CLASE STRING

- ▶ Clase String:

- ▶ Ejercicios:

- ▶ Crear un String con la siguiente descripción: "Soy un String en Java":
- ▶ Obtener la cantidad de caracteres que contiene dicho String.
- ▶ Crear otro String con la siguiente descripción: "Soy un string en java" y comparar ambos String
- ▶ Volver a comparar los String pero con el método equalsIgnoreCase();
- ▶ Obtener la palabra "String" del primer String y "java" del segundo.
- ▶ Convertir los siguientes numero enteros en String 4 y 10.
- ▶ Reemplazar los espacios por coma (,) y separar las palabras que contienen el String.

CLASE DE UTILIDAD

Math

Several thin, white, parallel lines are drawn diagonally across the right side of the image, starting from the top right and extending towards the bottom left.

CLASE STRING Y MATH

- ▶ Clase Math:

- ▶ Esta clase proporciona métodos para la realización de las operaciones matemáticas mas habituales en un programa. **Todos los métodos de la clase Math son estáticos**, por lo que no necesitamos crear ninguna instancia de la clase para su utilización, de hecho, no es posible crear ningún objeto de la clase **Math**.

CLASE STRING Y MATH

► Clase Math:

► Principales métodos:

- double **ceil(double n)**. Devuelve el entero mayor mas cercano al numero indicado en el parámetro. Ejemplo: `double d = Math.ceil(4.3)` //Retorna el valor 5.0
- double **floor(double n)**. Devuelve el entero menor mas cercano al numero indicado en el parámetro. Ejemplo: `double d = Math.floor(4.3)` //Retorna el valor 4.0
- double **round(double n)**. Devuelve el entero mas cercano al numero indicado en el parámetro. Ejemplo: `double d = Math.round(4.3)` //Retorna el valor 4.0
- double **pow(double n1, double n2)**. Devuelve el resultado de elevar n1 a n2.
- double **random()**. Devuelve un numero aleatorio mayor o igual que 0.0 y menor que 1.0

CLASE STRING Y MATH

- ▶ Clase Math:

- ▶ Ejercicios

- ▶ Realizar el siguiente calculo 2^8
- ▶ Obtener un numero random entre 0.0 y 1.0
- ▶ Obtener un numero random entre 0 y 15

CLASE DE UTILIDAD

Fechas

UTILIZACIÓN DE FECHAS

- ▶ El paquete **java.util** proporciona dos clases para el tratamiento básico de fechas en una aplicación Java, se trata de las clases **Date** y **Calendar**.

- ▶ Clase Date:

Un objeto Date representa una fecha y hora concretas con precisión de un milisegundo. Dicha clase permite manipular una fecha y obtener información de la misma de una manera sencilla.

Aunque la mayoría de los métodos de esta clase estén **deprecated** (obsoletos), no esta demás conocer su funcionamiento básico.

UTILIZACIÓN DE FECHAS

► Clase Date:

Utilizando el constructor por defecto de la clase **Date**, obtendremos un objeto **Date** representando la fecha y hora actuales.

A través del método **toString()**; obtendremos la representación en forma de cadena de caracteres de la fecha y hora actuales. Por ejemplo:

```
public static void main (String[] args) {  
    Date dt = new Date();  
    System.out.println("Fecha: " + dt.toString()) ;  
}
```

UTILIZACIÓN DE FECHAS

► Clase Date:

Lo que hace internamente el objeto **Date** es guardar la información de la fecha y hora como un numero de tipo **long** que representa la cantidad de milisegundos transcurridos desde el 01-01-1970 hasta el momento de su creación. Este numero puede ser obtenido a través del método **getTime()**:

Sabiendo como es su funcionamiento, podremos usar una simple sobrecarga sobre el constructor para generar un objeto **Date**. Por ejemplo:

```
public static void main (String[] args) {  
    Date dt = new Date(200000000000000L);  
    System.out.println("Fecha: " + dt.toString()) ;  
}
```

UTILIZACIÓN DE FECHAS

- ▶ Clase Calendar:
- ▶ Dicha clase surgió para cubrir las carencias de la clase Date.
- ▶ Es una **clase abstracta**, por lo cual no podremos instanciar un objeto de dicha clase usando el operador **new**
- ▶ Para la creación de objeto **Calendar**, utilizamos su método estático **getInstance()**:
- ▶ Utilizando el método **get()** obtendremos de manera individual cada uno de los campos que componen una fecha.
- ▶ Dicha clase define una serie de constantes con los valores que corresponden a cada uno de los campos de una fecha.

UTILIZACIÓN DE FECHAS

- ▶ Clase Calendar:

- ▶ Ejemplo:

```
public static void main (String[] args){  
    Calendar calendar = Calendar.getInstance();  
    String fechaPrint = "";  
    fechaPrint = calendar.get(Calendar.DAY_OF_MONTH) + "/" +  
                  (calendar.get(Calendar.MONTH) + 1) + "/" +  
                  calendar.get(Calendar.YEAR);  
    System.out.println("Hoy es: " + fechaPrint);  
}
```

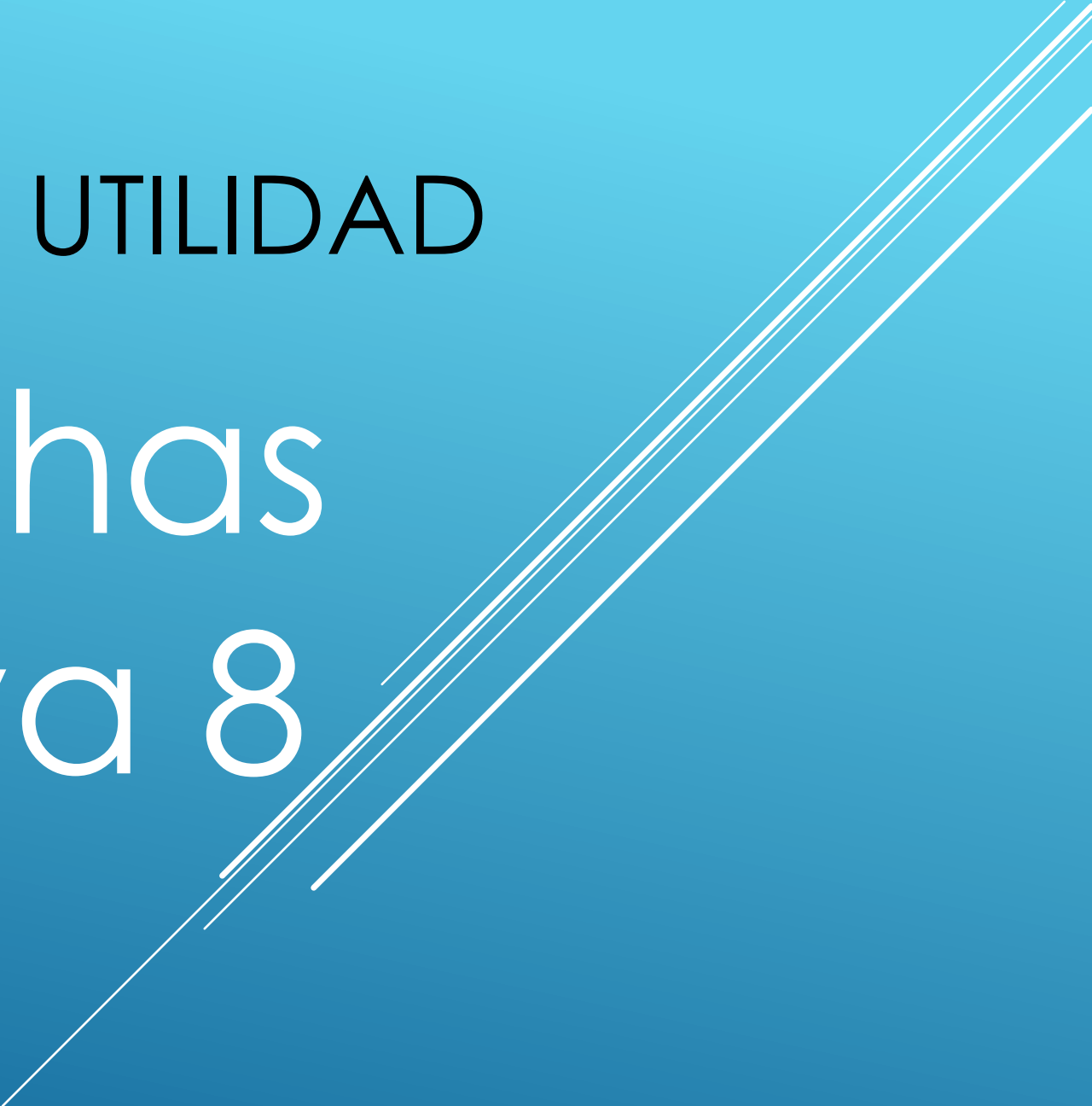
- ▶ En el caso del mes el valor retornado por **get()** esta comprendido entre 0 y 11.

UTILIZACIÓN DE FECHAS

- ▶ Métodos de la Clase Calendar:
- ▶ `void set(int año, int mes, int día):` Modifica la fecha del objeto Calendar.
- ▶ `void setTime(Date date):` Establece la fecha y hora del objeto Calendar a partir de un objeto Date.
- ▶ `Date getTime():` Devuelve la fecha/hora como un objeto Date.
- ▶ `void add(int campo, int cantidad):` Realiza una modificación relativa en uno de los campos de la fecha/hora, añadiendo una cantidad de tiempo al campo especificado. Por ejemplo: **`calendar.add(Calendar.MINUTE, 10);`**

CLASE DE UTILIDAD

Fechas Java 8

Several thin, white, parallel diagonal lines crossing the bottom right portion of the slide.

UTILIZACIÓN DE FECHAS EN JAVA 8

- ▶ Nueva API de fechas:
- ▶ *A partir de la versión 8 de Java se genero una nueva API para el manejo de fechas. Esto no implica que no se puedan utilizar las clases **Date** y **Calendar**.*
- ▶ La nueva API se encuentra dentro del paquete **java.time**
- ▶ Esta nueva API fue desarrollada para facilitar el manejo de fechas, tiempos, periodos y duraciones de tiempo.

UTILIZACIÓN DE FECHAS EN JAVA 8

► Clases y Enumerados:

► **Clases mas importantes:**

Instant: es un timestamp numérico, util para el uso de log y Framework de persistencia.

LocalDate: almacena la fecha sin hora.

LocalTime: almacena la hora sin fecha.

LocalDateTime: almacena la fecha mas la hora.

ZonedDateTime: almacena la fecha mas la hora, con información de uso horario.

► **Enumeradores:**

DayOfWeek: contiene los días de la semana, ejemplo: Lunes, Martes, etc.

Month: contiene los meses del año, ejemplo Enero, Febrero, etc.

UTILIZACIÓN DE FECHAS EN JAVA 8

- ▶ Enumerados:
- ▶ **DayOfWeek:** contiene constantes que definen cada uno de los días que contiene la semana. Para utilizar uno de estos enumeradores se realiza de la siguiente manera **DayOfWeek.Constante**, donde Constante puede ser **FRIDAY, MONDAY**, etc.
- ▶ **Month:** contiene constantes que definen cada uno de los meses que contiene el año. Para utilizar uno de estos enumeradores se realiza de la siguiente manera **Month.Cosntante**, donde Constante puede ser **JANUARY, FEBRUARY**, etc.
- ▶ **Ejercicio:** imprimir por pantalla lunes y enero utilizando estos enumeradores.

UTILIZACIÓN DE FECHAS EN JAVA 8

► Clases:

- **LocalDate:** nos permite instanciar un objeto **LocalDate** a partir de una fecha que se asigne o a partir de la fecha actual, esto se realiza a través de sus métodos **of(int año, int mes, int diaDelMes)** o el método **now()**.
- **LocalTime:** nos permite instanciar un objeto **LocalTime** a partir de un tiempo que se asigne o a partir del tiempo actual, esto se realiza a través de sus métodos **of(int hora, int minutos, int segundos)** o el método **now()**.
- **Ejercicio:**
 - 1 – Imprima por pantalla la fecha actual y el tiempo actual por separado.
 - 2 – Imprima por pantalla una fecha y tiempo cualquiera por separado.
 - 3 – Imprima por pantalla la fecha y tiempo actual juntos.

Enumeradores

Several thin, white, parallel diagonal lines crossing the lower right portion of the slide.

enumeradores

- ▶ Enumeradores (Enum):
- ▶ Fueron incluidos en la versión 1.5 de Java, consisten en la posibilidad de definir nuevos tipos de datos cuyos posibles valores están limitados a un determinado conjunto dado.
- ▶ Es una clase especial que a diferencia de las clases estándar no permiten que sean instanciadas.
- ▶ Limitan la creación de objetos, ya que se única limitación respecto a una clase normal es que el constructor del Enum debe ser de acceso privado.
- ▶ Son constantes inmutables, ya que su constructor es privado y no permite que se instancien los mismos.
- ▶ Pueden ser comparados en expresiones de comparación mediante el signo ==.

enumeradores

- ▶ Enumeradores (Enum) – Creación:

- ▶ Su creación es sencilla al igual que cualquier clase normal en Java.

```
public enum TipoPizarron {  
    BLANCO, NEGRO  
}
```

- ▶ Alguno de sus métodos:

- ▶ **String name()**: retorna un String con el nombre de la constante.
- ▶ **Int ordinal()**: retorna un entero con la posición del enum, según lo declarado.
- ▶ **Enum[] values()**: retorna un array del tipo del enum que contine todas sus constantes.

enumeradores

► Enumeradores (Enum) – Creación:

```
public enum TipoPizarron {  
    BLANCO(1, "Blanco"),  
    NEGRO(2, "Negro");  
  
    private int key;  
    private String value;  
  
    private TipoPizarron(int key, String value) {  
        this.key = key;  
        this.value = value;  
    }  
  
    public int getKey() {  
        return key;  
    }  
  
    public String getValue() {  
        return value;  
    }  
}
```

contengan distintos tipos primitivos.

```
public static void main(String[] args) {  
    TipoPizarron tPi = TipoPizarron.BLANCO;  
    System.out.println("Key: " + tPi.getKey());  
    System.out.println("Value: " + tPi.getValue());  
}
```

